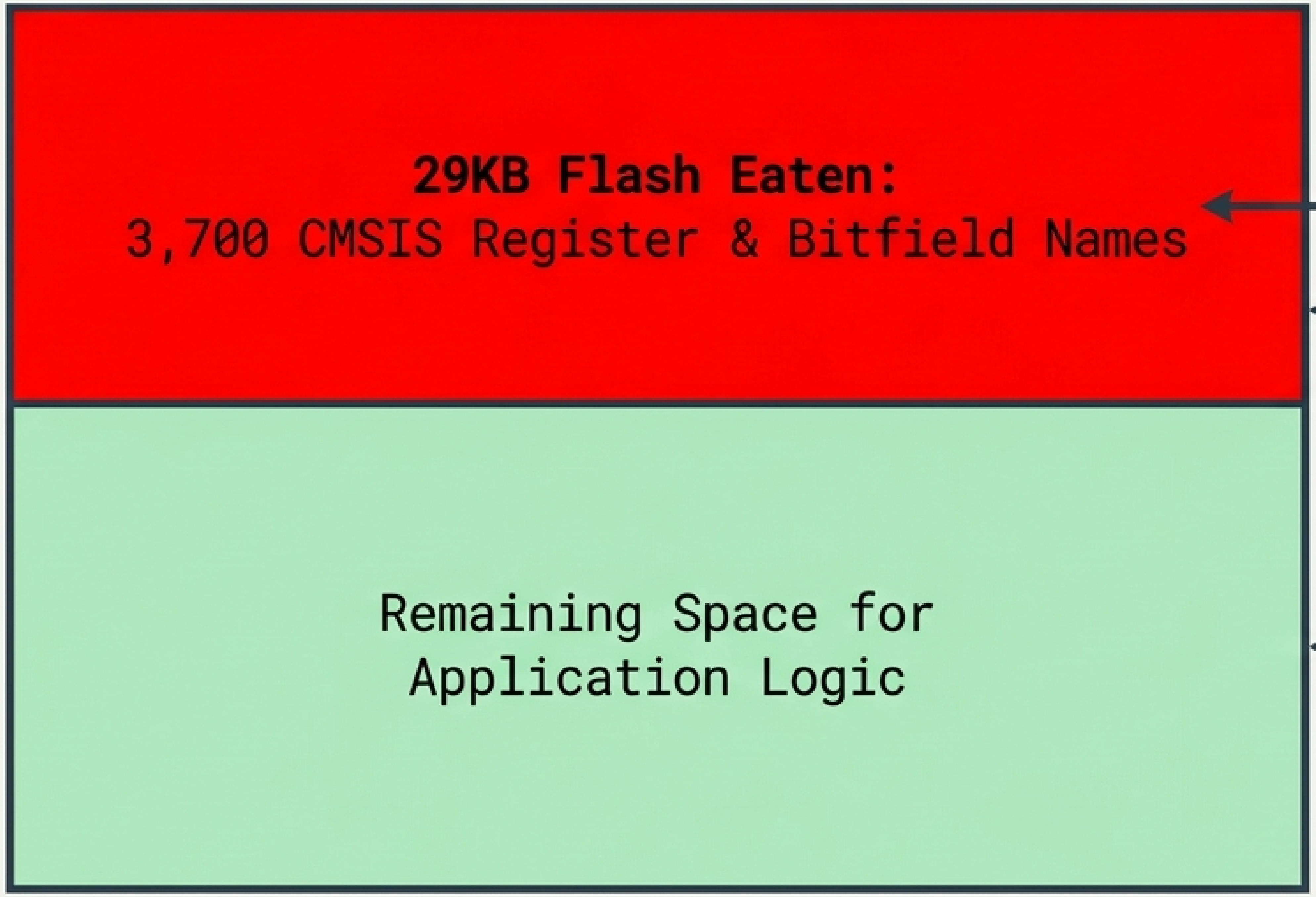
# FURS: Write Readable Forth, Pay Zero Flash Tax.

A build pipeline that keeps the CMSIS dictionary on your host, delivering pure absolute addresses to the target.

**GPIOC_BSRR_BS9** → **$48000818**

# Standard CMSIS Names Carry a 45% Readability Tax

STM32F051 Flash Memory Map: 64KB Total



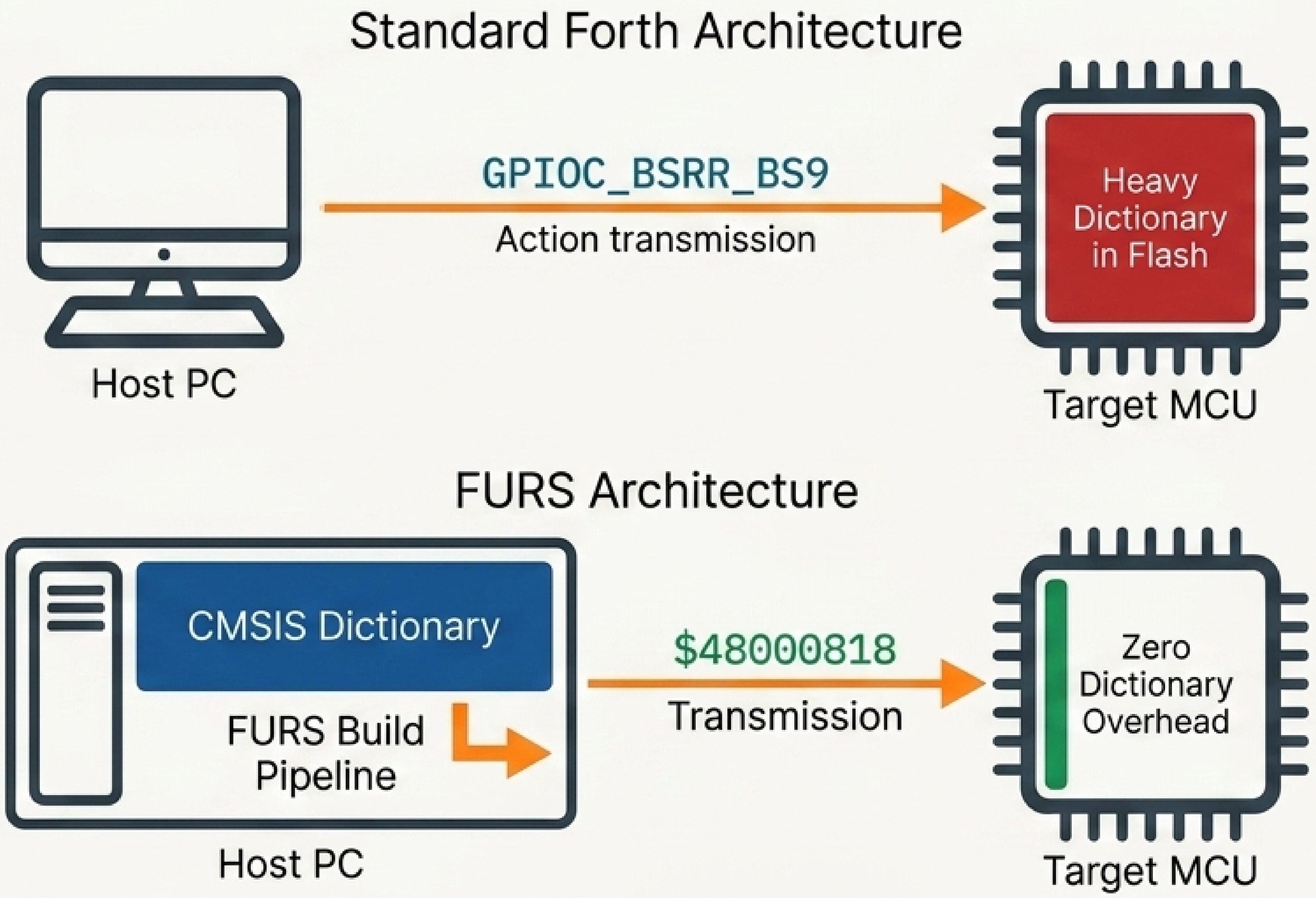

Mecrisp-Stellaris Forth stores its dictionary directly on the chip.

If you want standard ARM CMSIS names, you pay for them in flash before writing a single line of logic.

The choice: Write cryptic hex, or lose half your memory.

# The Pivot: Shift the Dictionary to the Host Machine

FURS (Forth Upload Replacement System) resolves human-readable names into absolute memory addresses at build time. The names never touch the microcontroller.



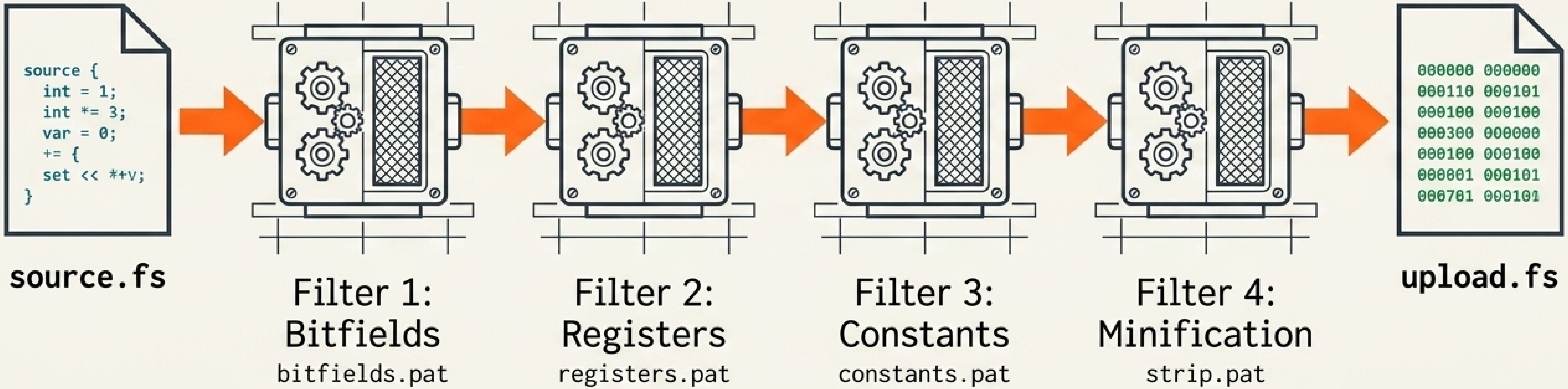
# Standard Forth vs. The FURS Paradigm

| | Standard Forth | FURS Paradigm |
|---|---|---|
| Dictionary Location | On-Target (Flash) | On-Host (Build Machine) |
| Code Readability | Low (Raw Hex) OR High (Flash Sacrificed) | High (Standard CMSIS Names) |
| Flash Overhead | Up to 29KB | 0 Bytes |
| Execution | Dictionary Lookup Required | Pure Absolute Hardware Addresses |

FURS applies the symbolic resolution of a C compiler to an interactive Forth environment.

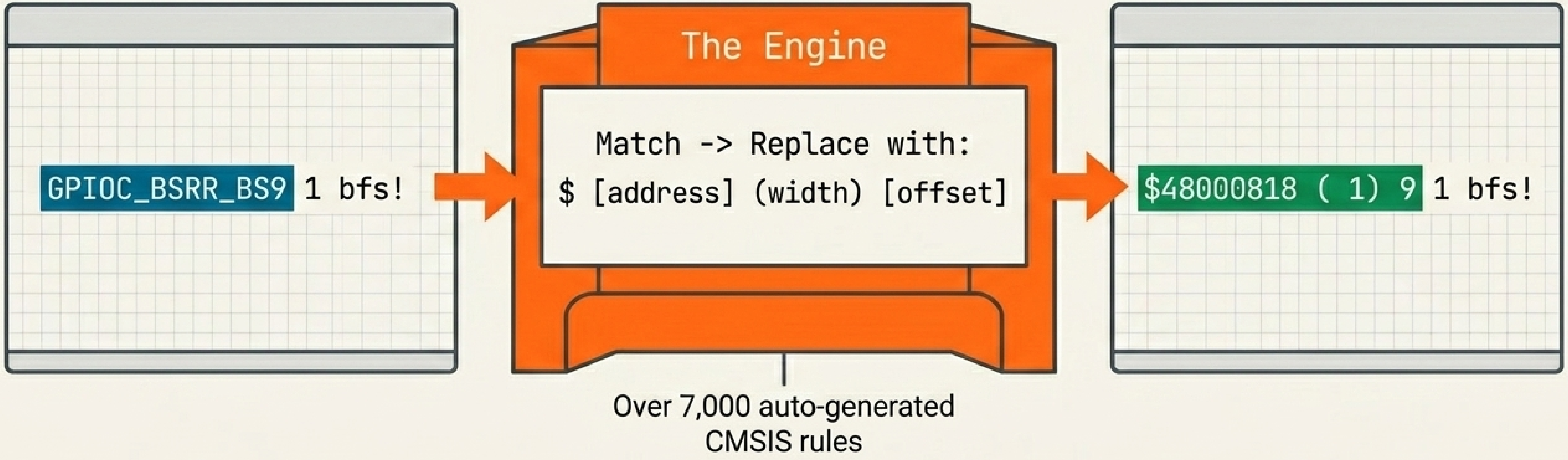
# The 4-Stage Gema Filtration Pipeline

Powered by the lightweight gema text processor, FURS systematically strips human syntax, substituting it with absolute binary instructions.



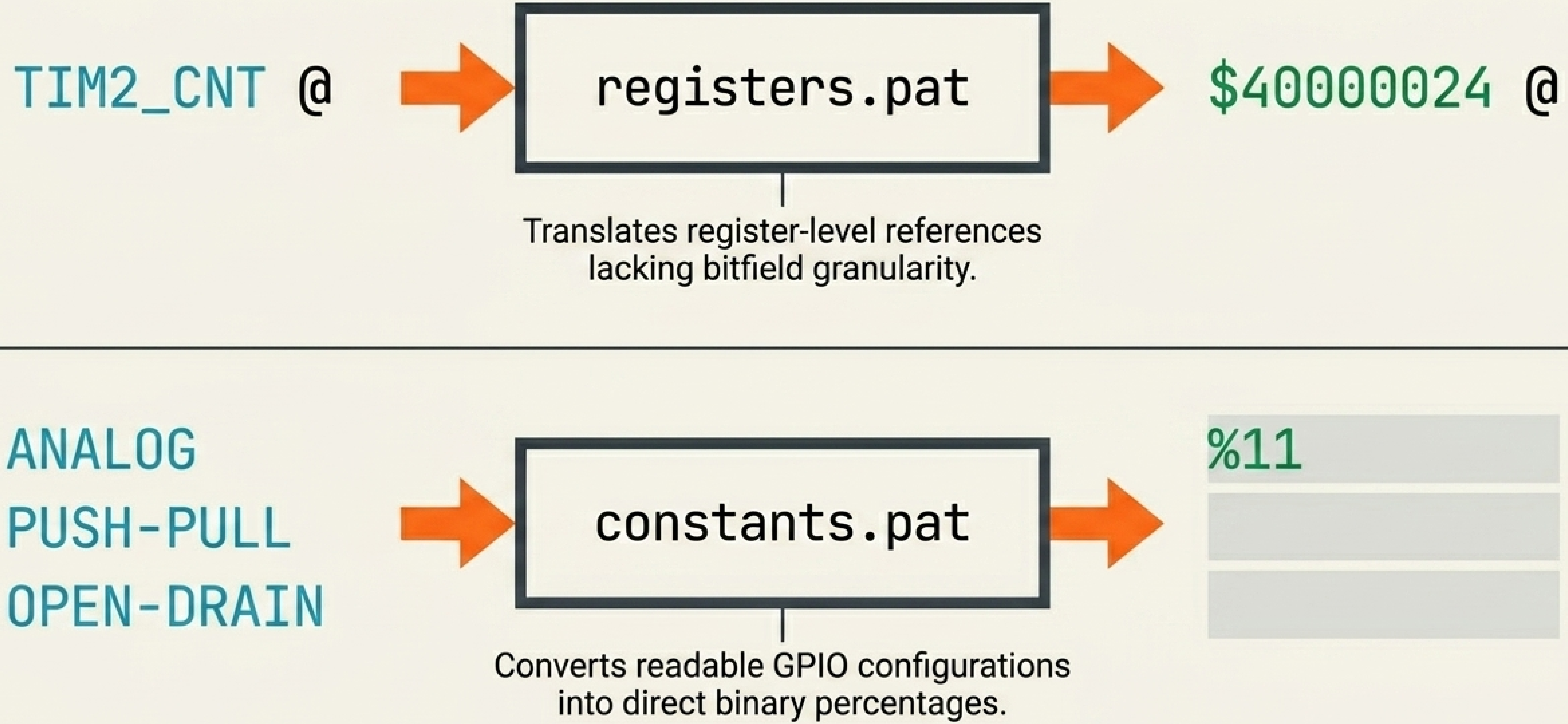
# Stage 1: Atomic Bitfield Resolution

Friendly names are translated into exact bit-manipulation parameters. The `bfs!` (bit-field set) utility word then executes an atomic write using these raw values.

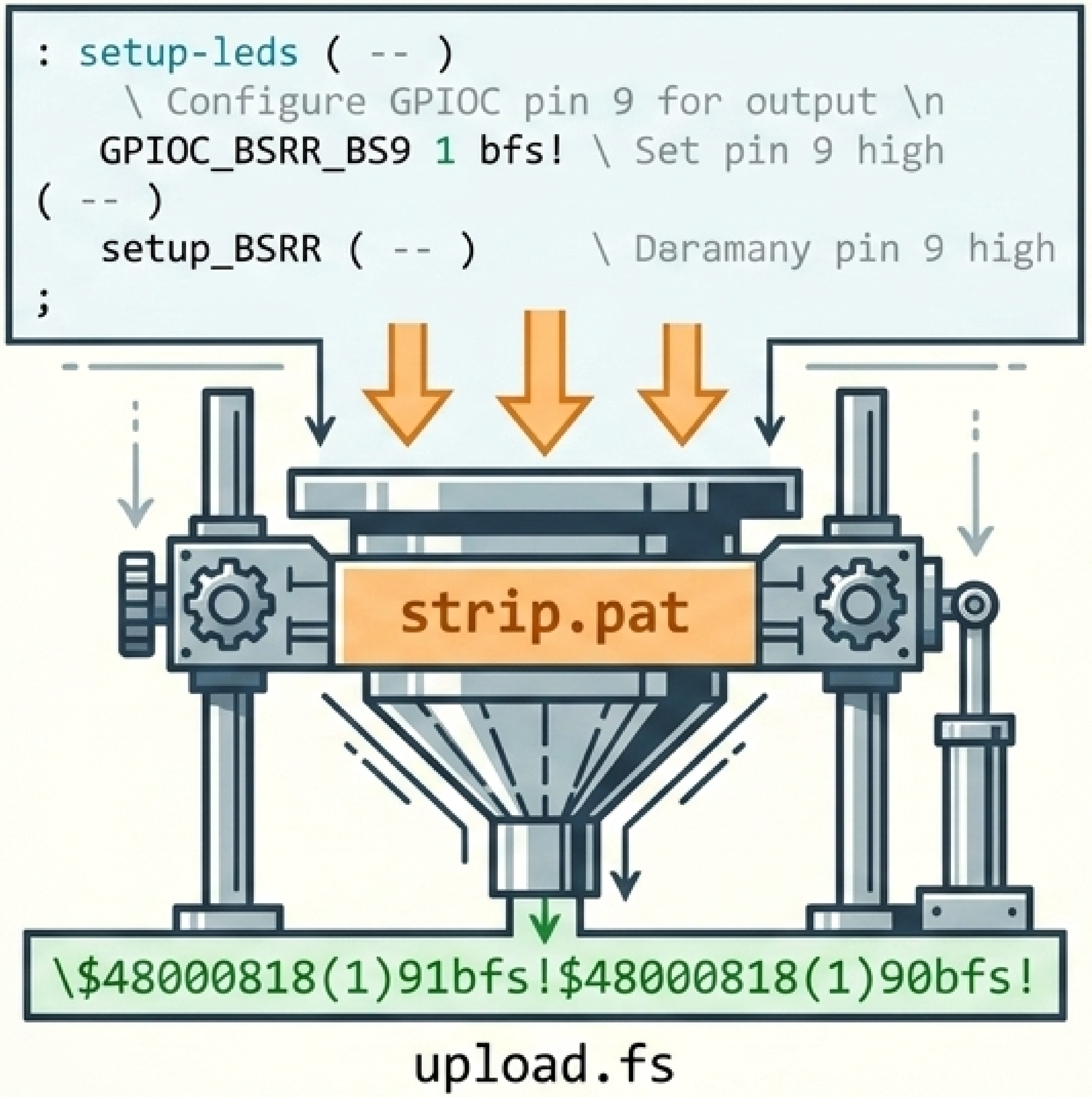



NotebookLM

# Stages 2 & 3: Resolving Registers and Magic Numbers



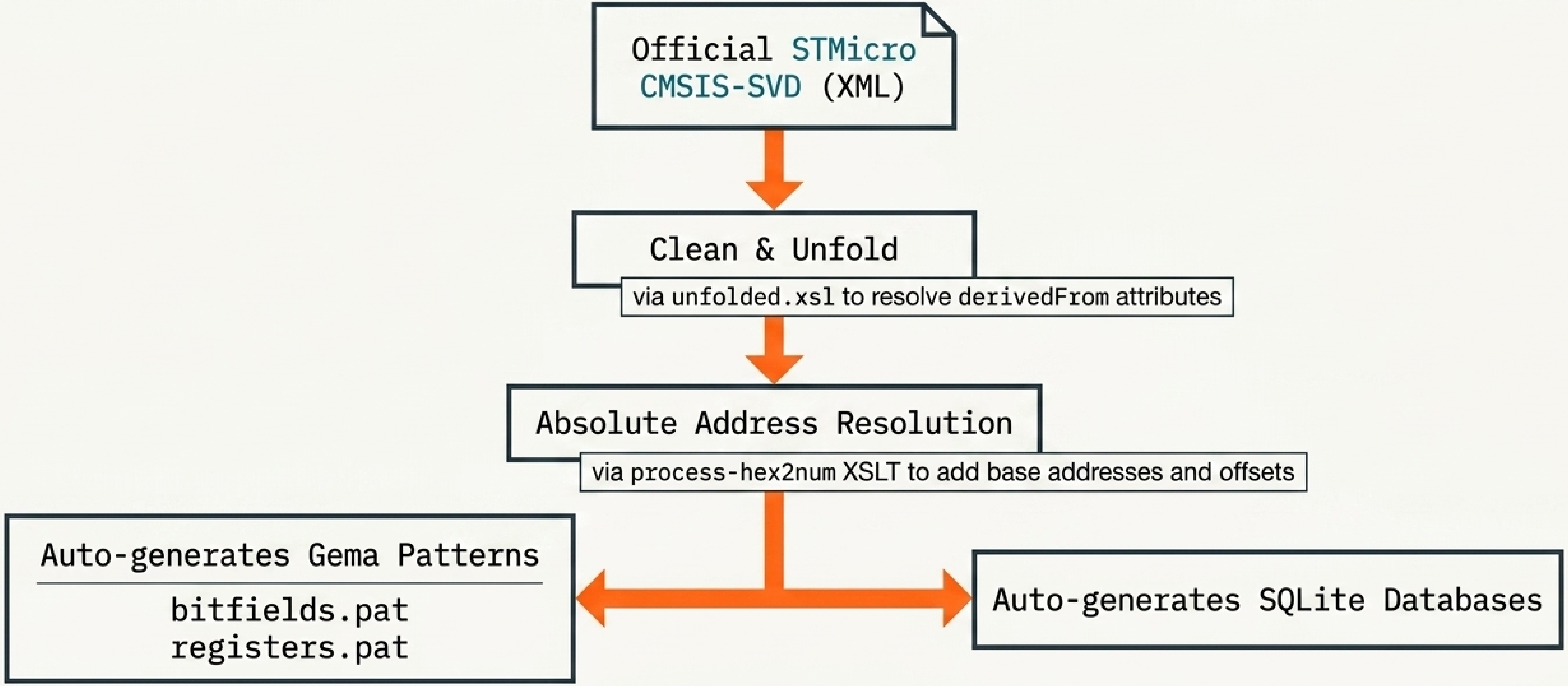
# Stage 4: Stripping Human Elements for Dense Binaries




Mecrisp-Stellaris doesn’t need your comments or stack notations. The final stage removes all whitespace and formatting, producing a minimal artifact ready for instant flashing.

# The Engine Room: Parsing Official SVD Files

FURS is inherently chip-agnostic. It algorithmically calculates absolute addresses from standard SVD files, generating the substitution rules automatically.

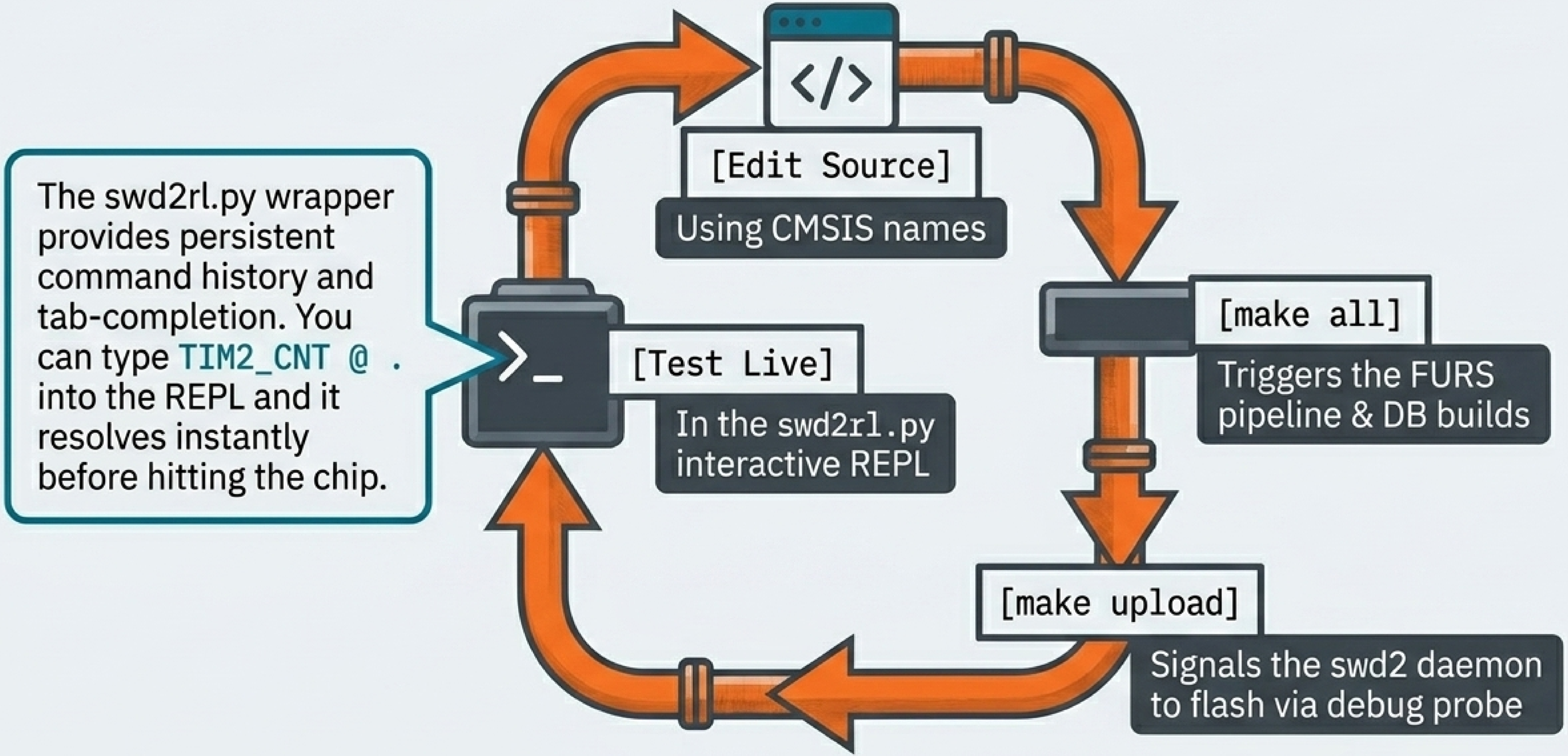


# An Interactive Memory Twin in the Terminal

```
$ furs-cli-relational
  simulate SQL query as furs-cli-relational
+------------+---------------+-------------+---------------+
| PERIPHERAL | REGISTER      | FIELD       | DESCRIPTION   |
+------------+---------------+-------------+---------------+
| RTC        | RTC_TR (0x00) |             | Time Register |
|            |               | HU [23:20]  | Hour units    |
|            |               | HT [19:16]  | Hour tens     |
|            |               | MNU [15:12] | Minute units  |
|            |               | MNT [11:8]  | Minute tens   |
|            |               | SU [7:4]    | Second units  |
|            |               | ST [3:0]    | Second tens   |
+------------+---------------+-------------+---------------+
| RTC        | RTC_DR (0x04) |             | Date Register |
+------------+---------------+-------------+---------------+
| ...        | ...           | ...         | ...           |
```

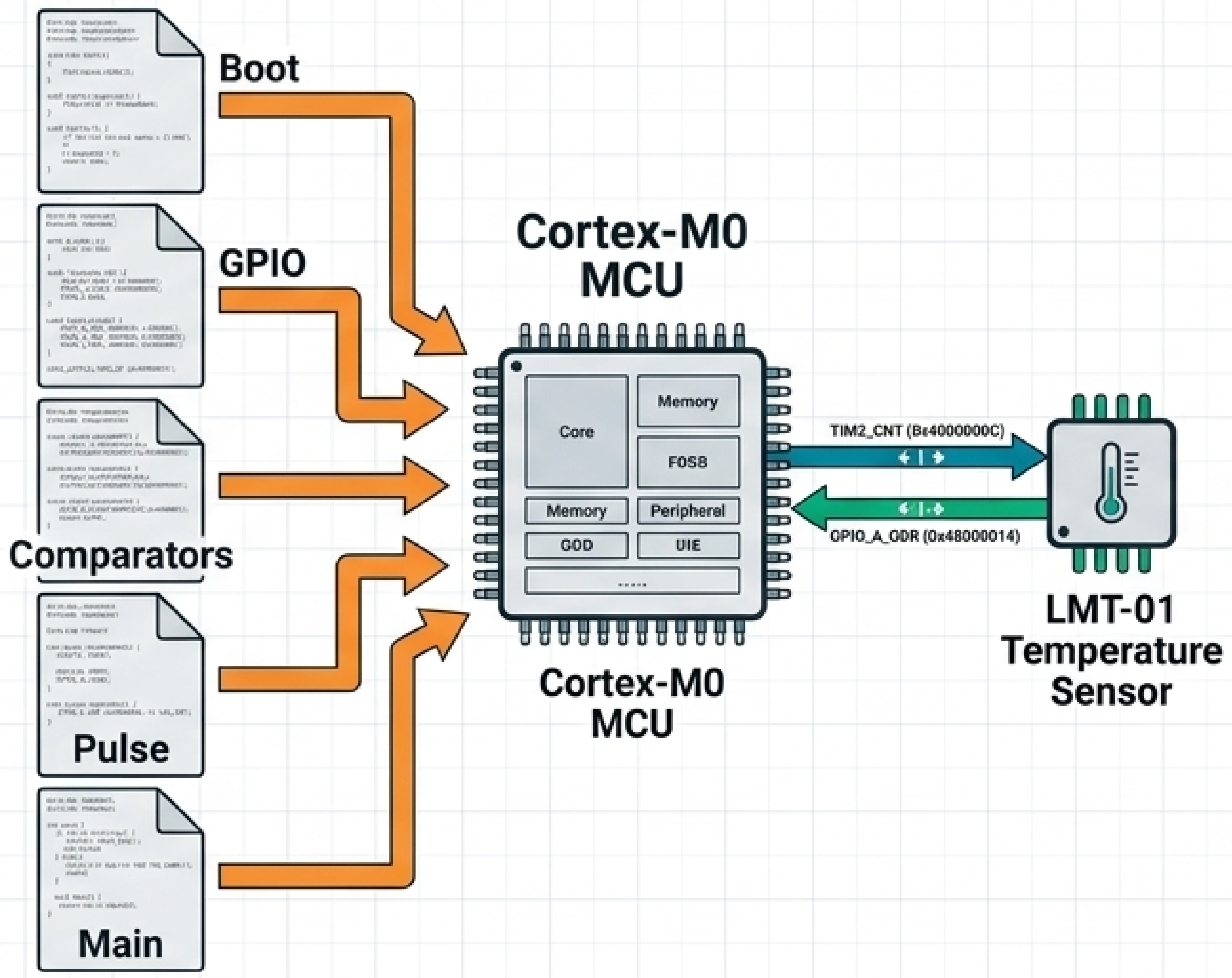
The build process parallel-generates relational and flat SQLite databases. You can instantly query the chip's exact memory map, search by peripheral, or find bits by description—without ever leaving the terminal.

# A Seamless, REPL-Driven Development Loop



# Real-World Proof: The LMT-01 Sensor Application

A complete application handling GPIO, Timers, and Comparators. It reads like an STMicroelectronics application note, but compiles with the density of raw assembly.




**200**
Lines of Code

**6**
Source Files

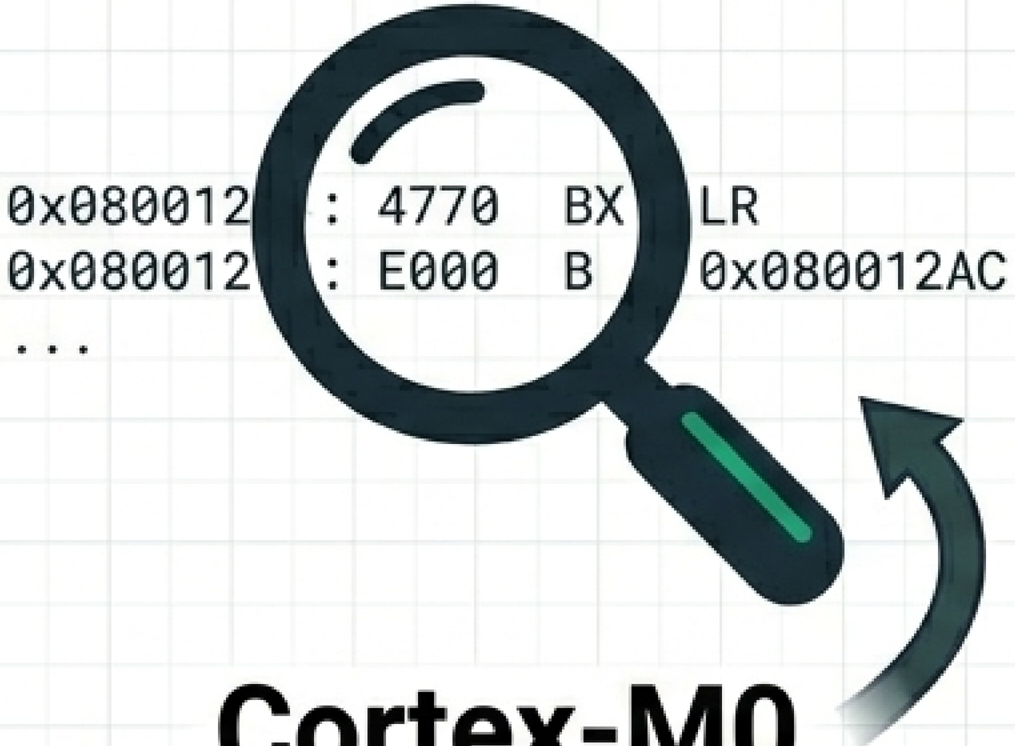
**~6KB**
Total Flash Footprint

**0 Bytes**
Flash Cost for CMSIS Names
(Zero overhead for human-readable names)

NotebookLM

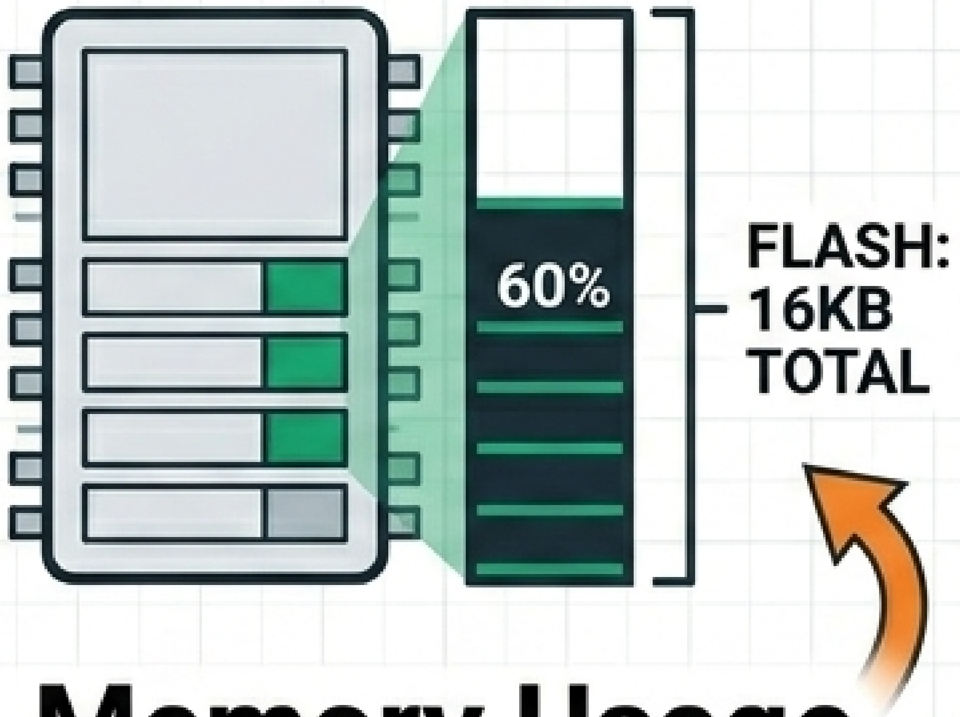
# Essential Embedded Tooling Shipped Standard

The core library ships with embedded lifesavers that every Forth programmer eventually needs, built right in.




## Cortex-M0 Disassembler

Inspect compiled machine code directly on the target.




## Memory Usage Stats

Read the chip's exact flash size register and track remaining capacity.

## Calltrace Handler

A vital debugging tool that dumps the return stack if an unhandled interrupt fires on a **JTAG-less** chip.

# A Radically Lightweight Host Stack

Beautifully simple infrastructure. If you have a Unix system and a C compiler, you can build and run the entire FURS pipeline.

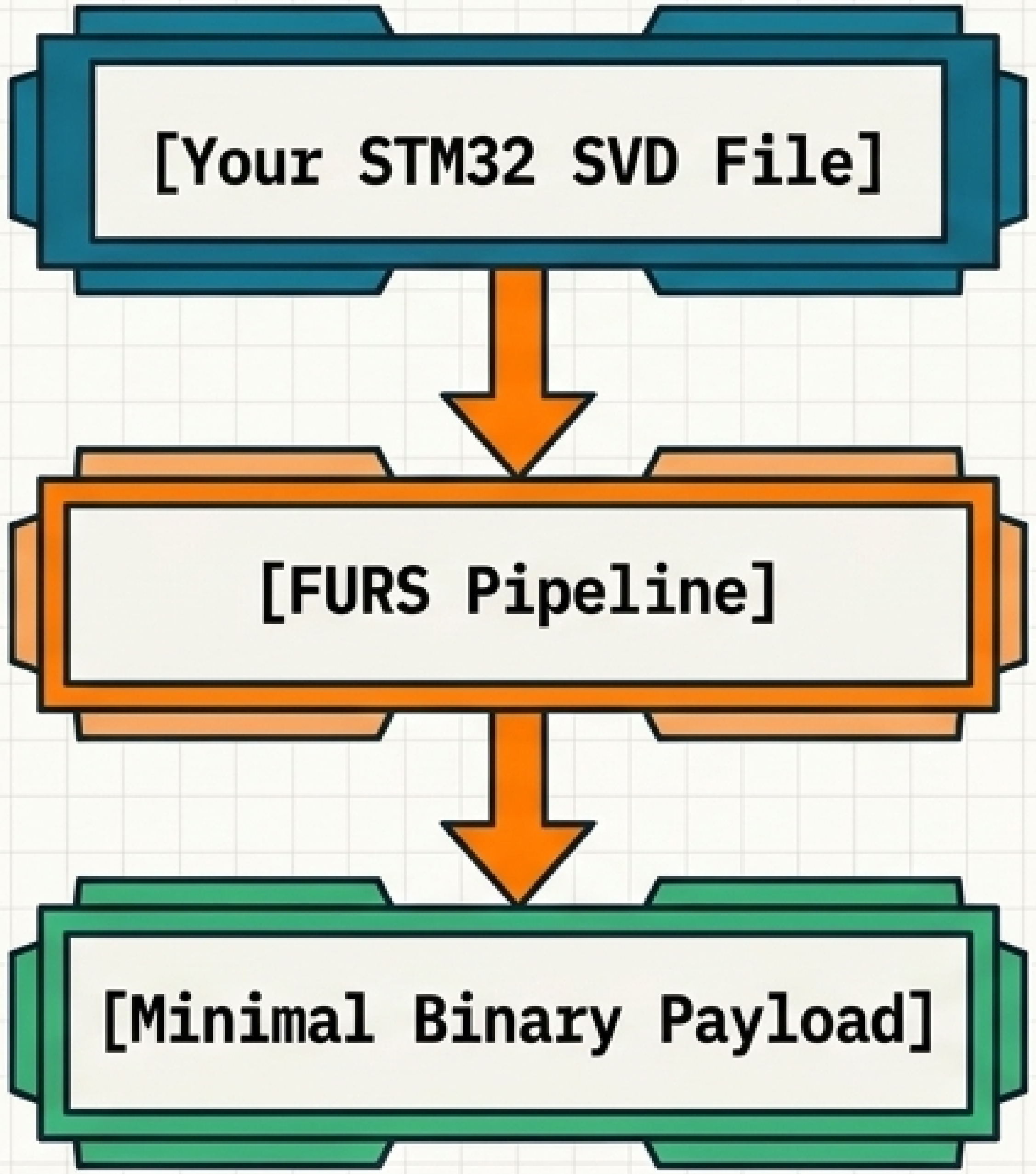
| | |
|---|---|
| `gema` (A single-file portable C program) | `xsltproc` (Standard libxslt installation) |
| `sqlite3` (Zero-configuration database) | `make` (Standard build coordinator) |

**NO Python venvs.**

**NO Node modules.**

**NO Docker.**

# Write Like Documentation, Compile Like Assembly.




- **Zero Flash Tax:** Reclaim up to 45% of your MCU's memory by processing dictionaries on the host.
- **Chip Agnostic:** Swap the SVD file to generate a completely new target registry instantly.
- **No Compromises:** Enjoy tab-completed CMSIS names in a live REPL without paying the embedded penalty.

Drop your SVD file into the project root, run make all, and start building.